

AWS SAM

Serverless Application Model

AWS Certified Developer Associate

Instructor: Marlon Anesi

O Que É AWS SAM?

Definição

Framework open-source para construir aplicações serverless

Extensão do CloudFormation focada em serverless

Componentes do SAM:

- SAM Template: sintaxe simplificada (YAML)
- SAM CLI: ferramenta de linha de comando
- Build, test e deploy localmente

Por que usar SAM?

- Menos código que CloudFormation puro (até 10x)
- Teste e debug local de funções Lambda
- Integração nativa com CodeDeploy para deploys seguros
- Templates portáteis e reutilizáveis

Recursos SAM Principais

AWS::Serverless::Function

- Define função Lambda com configurações simplificadas
- Eventos embutidos: API, S3, DynamoDB, SQS

AWS::Serverless::Api

- Cria API Gateway REST automaticamente
- CORS, autorização e estágios configuráveis

AWS::Serverless::SimpleTable

- Tabela DynamoDB com apenas uma partition key
- Ideal para casos simples

AWS::Serverless::HttpApi

- HTTP API (mais rápido e barato que REST API)
- OAuth 2.0 e JWT nativos

Estrutura de um Template SAM

Exemplo de template mínimo:

Transform: AWS::Serverless-2016-10-31

Globals:

Function:

Runtime: python3.11

Timeout: 30

Resources:

HelloFunction:

Type: AWS::Serverless::Function

Properties:

Handler: app.lambda_handler

Events:

HelloApi:

Type: Api

Properties:

Path: /hello

Method: get

Casos de Uso e Boas Práticas

APIs Serverless

- API Gateway + Lambda
- Autenticação Cognito
- CORS configurado
- Deploy gradual canary

Processamento de Eventos

- S3 trigger para Lambda
- SQS queue para batch
- EventBridge schedulers
- DynamoDB Streams

Boas Práticas:

- Use Globals para configurações comuns entre funções
- Organize código em camadas (Layers) para dependências
- Configure políticas IAM específicas por função
- Use sam local para testar antes do deploy
- Habilite X-Ray tracing para monitoramento

Pontos-chave: SAM Fundamentos

1. O que é SAM

- Framework open-source para serverless
- Extensão do CloudFormation
- Sintaxe simplificada (até 10x menos código)

2. Recursos principais

- `Serverless::Function` (Lambda)
- `Serverless::Api` (API Gateway REST)
- `Serverless::HttpApi` (HTTP API)

Pontos-chave: SAM Template e CLI

3. Template SAM

- Requer Transform: AWS::Serverless-2016-10-31
- Globals define configurações compartilhadas
- Events embutidos no próprio recurso

4. Vantagens SAM

- Teste local com `sam local invoke`
- Deploy gradual com CodeDeploy integrado
- Menos código, mais produtividade

LEMBRE-SE: Se questão menciona 'aplicação serverless' ou 'Lambda + API Gateway' > pense AWS SAM!